

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D1 — Design Specification Document

Part 2 Submission

22 May 2026

Halit Batuhan ASLAN

220104004003

Design Specification Contents

[Language Overview\[P1\]](#)

[Lexical Structure\[P1\]](#)

[Syntax\[P1\]](#)

[Semantics\[P2\]](#)

[Type System\[P2\]](#)

[Names, Binding, Scope, Lifetime\[P1\]](#)

[Expression & Assignment\[P2\]](#)

[Design Rationale\[P2\]](#)

Language Overview;

My language name is "Ayar", which corresponds to an old Turkish word meaning adjustment and regulation. This name directly reflects the language's purpose: to set and tune hyperparameters for machine learning models and present the user with the best option. The domain is ML model configuration, training simulation, and best-model selection. Ayar does not run real ML libraries; it simulates the training process and metric generation internally. One sample program is ;

```
// Dataset

dataset iris = load("iris.csv");

split iris into train(0.70), validation(0.15), test(0.15);

// Models

model KNN knn_small
{
    k = 3,
    distance = "euclidean"
}

model DecisionTree dt_main
{
    max_depth = 5,
    criterion = "gini"
}

// Experiment

experiment iris_exp
{
    train knn_small on iris.train;
    train dt_main on iris.train;
    evaluate knn_small on iris.validation;
    evaluate dt_main on iris.validation;
    collect metrics [accuracy, f1];
    analyze overfitting
    {
        threshold = 0.10;
        underfit_max_acc = 0.70;
```

```

}

compare by f1 higher_is_better;

exclude if overfit or underfit;

select best;

}

// Final evaluation and report

evaluate iris_exp.best on iris.test;

report summary

{

    metrics = [accuracy, f1],

    show = overfitting_analysis

}

```

My language prioritizes readability (Sebesta §1.3.1) because I believe reading and reviewing code later is far more important than writing it quickly. For cost (§1.3.4), readability directly reduces maintenance cost — in a typical ML project lifecycle, reading and reviewing code takes significantly more time than writing it, so optimizing for readability lowers the long-term cost of using the language. For reliability (§1.3.3), Ayar enforces strong typing where `metric ≠ float`, detects data leakage (training on the test set) as a runtime error, and uses name equivalence to prevent accidental type confusion — all of which reduce silent errors in model selection. Ayar knowingly sacrifices writability (§1.3.2): keywords like `higher_is_better` and `overfitting_analysis` are verbose, and every `{` must be placed on its own line. These rules make programs longer to type but significantly easier to read and review.

Lexical Structure;

A	B
Token Categories	Regular Expression or Pattern
INT_LIT	<code>[0-9]+</code>
FLOAT_LIT	<code>[0-9]+ \. [0-9]+</code>
STRING_LIT	<code>\("[^"\n]*\"</code>
BOOL_LIT	<code>true false</code>
OPERATOR	<code>* - + / < > ==</code>
SEPARATOR	<code>{ } ; : , ()</code>
COMMENT	<code>// [^\\n]* /* [^*]* */</code>
KEYWORD	<code>experiment, model, train etc.</code>
IDENTIFIER	<code>iris, knn_baseline etc.</code>

Syntax;

<program> ::= { <top_level_decl> }

<top_level_decl> ::= <dataset_decl> | <split_stmt> | <model_decl> | <experiment_decl> |
<eval_stmt> | <report_stmt>

(* — DATASET — *)

<dataset_decl> ::= "dataset" <identifier> "=" "load" "(" <string_literal> ")" ";"

<split_stmt> ::= "split" <identifier> "into" <split_part> "," <split_part> "," <split_part> ";"

<split_part> ::= <split_role> "(" <float_literal> ")"

<split_role> ::= "train" | "validation" | "test"

(* — MODEL — *)

<model_decl> ::= "model" <model_type> <identifier> "{" <field_list> "}"

<model_type> ::= "KNN" | "DecisionTree" | "SVM" | "NaiveBayes" | "LogisticRegression"

<field_list> ::= <field> { "," <field> }

<field> ::= <identifier> "=" <field_value>

<field_value> ::= <int_literal> | <float_literal> | <string_literal> | <bool_literal>

(* — EXPERIMENT — *)

<experiment_decl> ::= "experiment" <identifier> "{" { <experiment_stmt> } "}"

<experiment_stmt> ::= <train_stmt> | <evaluate_stmt> | <collect_stmt> | <analyze_stmt> |
<compare_stmt> | <exclude_stmt> | <select_stmt>

<train_stmt> ::= "train" <identifier> "on" <dataset_ref> ";"

<evaluate_stmt> ::= "evaluate" <identifier> "on" <dataset_ref> ";"

<dataset_ref> ::= <identifier> "." <split_role>

<collect_stmt> ::= "collect" "metrics" "[" <metric_list> "]" ";"

<metric_list> ::= <metric_literal> { "," <metric_literal> }

<analyze_stmt> ::= "analyze" "overfitting" "{" <analyze_field_list> "}"

<analyze_field_list> ::= <analyze_field> { <analyze_field> }

<analyze_field> ::= "threshold" "=" <float_literal> ";" | "underfit_max_acc" "=" <float_literal> ";"

<compare_stmt> ::= "compare" "by" <metric_literal> <compare_direction> ";"

<compare_direction> ::= "higher_is_better" | "lower_is_better"

<exclude_stmt> ::= "exclude" "if" <exclude_condition> ";"

<exclude_condition> ::= <exclude_term> { "or" <exclude_term> }

<exclude_term> ::= "overfit" | "underfit"

<select_stmt> ::= "select" "best" ";"

(* — TOP-LEVEL EVALUATE — *)

<eval_stmt> ::= "evaluate" <eval_target> "on" <dataset_ref> ";"

<eval_target> ::= <identifier> | <identifier> "." <identifier> | "[" <identifier> "." <identifier> "]"

(* — REPORT — *)

<report_stmt> ::= "report" <identifier> "{" <report_field_list> "}"

<report_field_list> ::= <report_field> { "," <report_field> }

<report_field> ::= "metrics" "=" "[" <metric_list> "]" | "show" "=" <show_value>

<show_value> ::= "overfitting_analysis" | <string_literal>

(* — EXPRESSIONS — *)

<expr> ::= <and_expr> | <expr> "or" <and_expr>

<and_expr> ::= <not_expr> | <and_expr> "and" <not_expr>

<not_expr> ::= <rel_expr> | "not" <not_expr>

<rel_expr> ::= <add_expr> | <add_expr> <rel_op> <add_expr>

<rel_op> ::= "==" | "!=" | "<" | "<=" | ">" | ">="

<add_expr> ::= <mul_expr> | <add_expr> "+" <mul_expr> | <add_expr> "-" <mul_expr>

<mul_expr> ::= <unary_expr> | <mul_expr> "*" <unary_expr> | <mul_expr> "/" <unary_expr>

<unary_expr> ::= <primary> | "-" <unary_expr>

<primary> ::= <int_literal> | <float_literal> | <bool_literal> | <string_literal> | <metric_literal>
| <identifier> "." <identifier> | <identifier> | "(" <expr> ")"

(* — LEXICAL — *)

<identifier> ::= <letter> { <letter> | <digit> | "_" }

<int_literal> ::= <digit> { <digit> }

<float_literal> ::= <digit> { <digit> } "." { <digit> } | "." <digit> { <digit> }

<bool_literal> ::= "true" | "false"

<string_literal> ::= "" { <string_char> } ""

<metric_literal> ::= "accuracy" | "precision" | "recall" | "f1"

<letter> ::= "a" | ... | "z" | "A" | ... | "Z"

<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<string_char> ::= (* any char except "" and unescaped newline *)

Sementics;

Semantics;

Two non-trivial constructs are formalized: (1) the experiment block — sequential execution with state accumulation across train, evaluate, collect, analyze, and compare statements; and (2) the compare-exclude-select pipeline — the core DSL-specific construct that ranks, filters, and selects the best model.

State Definition

Following Sebesta §3.5.2.2, the execution state of an Ayar program is a triple representing all information that changes as the program runs:

$$\sigma = (\Gamma, M, R)$$

Symbol	Type	Meaning
Γ	$\text{Name} \rightarrow \text{Value}$	Global environment: dataset and model names mapped to their declared values
M	$\text{ModelName} \rightarrow \text{MetricMap}$	Training results: each trained model mapped to its metric values
R	$\text{ModelName} \cup \{\perp\}$	Selection result: best model chosen by select best, or $\perp$ if not yet selected

A MetricMap is a partial function $\text{MetricName} \rightarrow \mathbb{R}$, where $\text{MetricName} \in \{\text{accuracy, precision, recall, f1}\}$. The distinction between M (training results for all models) and R (single best selection) directly reflects the two phases of the experiment pipeline: accumulation and selection.

Supporting Rules — Global Declarations

Before the experiment block can execute, datasets and their splits must be bound into Γ . The following two rules establish the preconditions required by E-Train and E-Evaluate.

Rule E-DatasetDecl

$\text{path} \in \text{String} \quad \text{id} \notin \text{dom}(\Gamma)$

$\langle \text{dataset id} = \text{load}(\text{path}), (\Gamma, M, R) \rangle$

$\rightarrow (\Gamma[\text{id} \mapsto \text{DatasetRecord}(\text{path})], M, R)$

Explanation: A dataset declaration binds id to a DatasetRecord in the global environment Γ . This rule establishes the precondition $\text{dataset_ref} \in \text{dom}(\Gamma)$ required by E-Train and E-Evaluate.

Rule E-SplitStmt

$\text{id} \in \text{dom}(\Gamma) \quad r_1 + r_2 + r_3 = 1.0$

$\langle \text{split id into train}(r_1), \text{validation}(r_2), \text{test}(r_3), (\Gamma, M, R) \rangle$

$\rightarrow (\Gamma[\text{id.train} \mapsto r_1, \text{id.validation} \mapsto r_2, \text{id.test} \mapsto r_3], M, R)$

*Explanation: The split statement partitions the dataset into three roles. This grounds the dotted references `iris.train`, `iris.validation`, `iris.test` used in *E-Train* and *E-Evaluate*, and establishes the role attribute checked by the data leakage guard `dataset_ref.role ≠ test` in *E-Train*.*

Construct 1 — The Experiment Block

Syntax: `experiment <id> { <stmt>* }`

An experiment block executes a sequence of statements, each of which transforms σ . The block opens an inner scope: names bound inside (specifically the implicit best selection) do not escape to the global environment Γ unless accessed via dotted notation (`exp_name.best`). This directly implements the stack-dynamic lifetime declared in §4.6: M is discarded after the block closes; only R is exported.

Notation: $\langle \text{stmt}, \sigma \rangle \rightarrow \sigma'$ means "statement `stmt` executed in state σ produces new state σ' ".

Rule E-Train

`model_name` $\in \text{dom}(\Gamma)$

`dataset_ref` $\in \text{dom}(\Gamma)$

`dataset_ref.role ≠ test` -- data leakage guard

`simulate`($\Gamma(\text{model_name})$, $\Gamma(\text{dataset_ref})$) = `metrics`

$\langle \text{train } \text{model_name} \text{ on } \text{dataset_ref}, (\Gamma, M, R) \rangle$

$\rightarrow (\Gamma, M[\text{model_name} \mapsto \text{metrics}], R)$

Explanation: Both the model and dataset must be declared in Γ . Training on the test split raises a runtime data-leakage error. The `simulate()` function produces a deterministic `MetricMap` for the given model configuration. The result is stored in M .

Rule E-Evaluate

`model_name` $\in \text{dom}(\Gamma)$ `model_name` $\in \text{dom}(M)$

`dataset_ref` $\in \text{dom}(\Gamma)$

`eval_metrics`($\Gamma(\text{model_name})$, $\Gamma(\text{dataset_ref})$, $M(\text{model_name})$) = `m'`

$\langle \text{evaluate } \text{model_name} \text{ on } \text{dataset_ref}, (\Gamma, M, R) \rangle$

$\rightarrow (\Gamma, M[\text{model_name} \mapsto m'], R)$

Explanation: Evaluation refines the metric values already stored for the model. The precondition `model_name` $\in \text{dom}(M)$ enforces that a model cannot be evaluated before it is trained — a semantic ordering constraint that cannot be expressed in the EBNF grammar alone.

Rule E-Collect

`metrics_list` = [`m1`, ..., `mn`] each `mi` $\in \{\text{accuracy, precision, recall, f1}\}$

$M' = \{ n \mapsto \text{restrict}(M(n), \text{metrics_list}) \mid n \in \text{dom}(M) \}$

$\langle \text{collect metrics } [\text{m}_1, \dots, \text{m}_n], (\Gamma, M, R) \rangle \rightarrow (\Gamma, M', R)$

Explanation: restrict(mm, list) keeps only the keys in list from each MetricMap. This narrows all models to only the metrics the user declared, preventing the pipeline from comparing on metrics that were not collected.

Rule E-Sequence

$$\langle s_1, \sigma_0 \rangle \rightarrow \sigma_1 \quad \langle s_2, \sigma_1 \rangle \rightarrow \sigma_2 \quad \dots \quad \langle s_n, \sigma_{n-1} \rangle \rightarrow \sigma_n$$

$$\langle s_1; s_2; \dots; s_n, \sigma_0 \rangle \rightarrow \sigma_n$$

Explanation: Following Sebesta §3.5 operational semantics style, statements execute left-to-right, each receiving the state produced by the previous one. This sequencing rule makes ordering semantically significant: evaluate cannot precede train because $M(model_name)$ would be undefined.

Rule E-ExperimentBlock

$$\langle s_1; \dots; s_n, (\Gamma, \emptyset, \perp) \rangle \rightarrow (\Gamma', M', R')$$

$$\text{exp_name} \notin \text{dom}(\Gamma)$$

$$\langle \text{experiment exp_name } \{ s_1; \dots; s_n \}, (\Gamma, M, R) \rangle$$

$$\rightarrow (\Gamma[\text{exp_name.best} \mapsto R'], M, \perp)$$

Explanation: The block starts with empty M ($\emptyset$) and no selection ($\perp$). After all statements execute, R' is exported to Γ under the dotted key exp_name.best . M' is not propagated — experiment-internal metric data is stack-dynamic and is discarded when the block closes. Only the best selection survives

Construct 2 — The Compare–Exclude–Select Pipeline

This is Ayar's core DSL-specific construct — the primary reason a generic scripting language cannot replace it. The three statements compare by, exclude if, and select best form a fixed-order pipeline that ranks, filters, and selects the best model. Pipeline input: M (all trained/evaluated models). Pipeline output: R (one model name, or a runtime error).

Rule P-AnalyzeOverfitting

$$\text{threshold} \in \mathbb{R}, \quad \text{underfit_max_acc} \in \mathbb{R}$$

for each $n \in \text{dom}(M)$:

$$\text{overfit}(n) \Leftrightarrow (M(n)(\text{train_acc}) - M(n)(\text{val_acc})) > \text{threshold}$$

$$\text{underfit}(n) \Leftrightarrow M(n)(\text{val_acc}) < \text{underfit_max_acc}$$

$$M_tagged = M \text{ with each name tagged } \{\text{overfit: Bool}, \text{underfit: Bool}\}$$

$$\langle \text{analyze overfitting } \{ \text{threshold}=t; \text{underfit_max_acc}=u; \}, (\Gamma, M, R) \rangle$$

$$\rightarrow (\Gamma, M_tagged, R)$$

Explanation: The analyze block computes Boolean tags for each model. No model is removed at this step — only tagged. The tags are consumed by the subsequent exclude statement. This separation of tagging and filtering is a deliberate design decision that makes the two concerns independently readable.

Rule P-Compare

$\text{metric} \in \text{dom}(M(n))$ for all $n \in \text{dom}(M)$

$\text{direction} \in \{\text{higher_is_better}, \text{lower_is_better}\}$

$\text{ranked} = \text{sort}(\text{dom}(M), \text{by: } \lambda n. M(n)(\text{metric}), \text{order: direction})$

$\langle \text{compare by metric direction}, (\Gamma, M, R) \rangle \rightarrow (\Gamma, M_{\text{ranked}}, R)$

Explanation: compare imposes a total ordering on models. The metric must be present in every model's MetricMap — guaranteed by the preceding collect statement. If direction = higher_is_better, models are sorted descending; lower_is_better gives ascending order. The ordering is consumed by select best.

Rule P-Exclude

$\text{conditions} \subseteq \{\text{overfit}, \text{underfit}\}$

$M' = \{ n \mapsto M(n) \mid n \in \text{dom}(M) \}$

$\wedge \neg(\text{overfit} \in \text{conditions} \wedge \text{overfit}(n))$

$\wedge \neg(\text{underfit} \in \text{conditions} \wedge \text{underfit}(n)) \}$

$\langle \text{exclude if } c_1 \text{ or } c_2, (\Gamma, M, R) \rangle \rightarrow (\Gamma, M', R)$

Explanation: exclude filters M, removing any model whose tags match any listed condition. The result M' contains only eligible models. If M' = ∅ (all models excluded), the subsequent select best raises a domain-specific runtime error: NoEligibleModelError.

Rule P-SelectBest (success case)

$\text{dom}(M') \neq \emptyset$

$\text{best_name} = \text{first}(\text{ranked}(M'))$

$\langle \text{select best}, (\Gamma, M', \perp) \rangle \rightarrow (\Gamma, M', \text{best_name})$

Rule P-SelectBest (failure case)

$\text{dom}(M') = \emptyset$

$\langle \text{select best}, (\Gamma, \emptyset, \perp) \rangle \rightarrow \text{NoEligibleModelError}$

Explanation: select best picks the first model in the ranked order from compare. The failure case is a deliberate design decision: Ayar fails loudly (NoEligibleModelError) rather than silently returning an overfit or underfit model. This directly supports the reliability criterion declared in §4.1 (Sebesta §1.3.3).

Type System;

1. Primitive Types — Value Sets (§6.1, §6.2)

Ayar has four primitive types:

int — whole numbers, value set $\mathbb{Z}$ (integers). Used for model hyperparameters like `k = 3`, `max_depth = 5`. No explicit range limit is declared; the interpreter treats them as arbitrary-precision integers.

float — real numbers, value set $\mathbb{R}$ (approximated). Used for split ratios like `train(0.70)` and overfitting thresholds like `threshold = 0.08`.

bool — value set `{true, false}`. Used in model field values and overfitting tag results (`overfit=true`).

metric — value set $\mathbb{R}$, but **name-distinct from float**. Represents ML evaluation metrics: accuracy, precision, recall, f1. Although the underlying representation is a real number, metric is a separate type — you cannot assign a metric value to a float variable or use it in general arithmetic.

2. Is Ayar Strongly Typed? (§6.12)

Yes, Ayar is strongly typed.

Ayar satisfies this in three specific ways:

- **metric $\neq$ float:** A metric literal (e.g. `accuracy`) cannot be used in arithmetic expressions (`accuracy + 0.5` is a type error). The types share the same underlying representation but are name-distinct — the type checker rejects any operation that crosses this boundary.
- **Model field type checking:** Each hyperparameter field is bound to a specific type at declaration time. Assigning a string to a field that expects an integer (`k = "three"`) is a type error caught before execution.
- **Dataset reference type checking:** `train` and `evaluate` statements accept only valid dataset references (`iris.train`, `iris.validation`, `iris.test`). Passing a model name where a dataset reference is expected is a type error.

3. Implicit Type Coercion (§7.4)

Ayar allows exactly one implicit coercion: `int` $\rightarrow$ `float` in mixed-mode arithmetic.

When an integer value appears where a float is expected — for example, in a split ratio computation or a threshold comparison — it is automatically widened to float. This is the only coercion rule.

All other type crossings are **not coerced** — they are type errors:

- `float` $\rightarrow$ `int` is never performed implicitly (no narrowing).
- `metric` $\rightarrow$ `float` is never coerced (name equivalence prevents it).
- `bool` $\rightarrow$ `int` is not allowed.

This follows the design of Java and other strongly typed languages (Sebesta §7.4): widen freely, never narrow silently.

4. Type Equivalence — Name or Structural? (§6.14)

Ayar uses name equivalence.

The most important case in Ayar is metric vs float. Both have the underlying value set $\mathbb{R}$. Under **structural equivalence**, they would be considered the same type — and `accuracy + 0.6` would silently compile. Under **name equivalence**, they are different types because they have different names, and the expression is a type error.

This choice is directly motivated by the language's reliability goal (§4.1, Sebesta §1.3.3): in ML model selection, confusing a raw float (e.g. a threshold value 0.06) with a metric result (e.g. `accuracy = 0.99`) is a silent, catastrophic error. Name equivalence forces the programmer to be explicit when crossing this boundary.

The trade-off accepted: more verbose code in cases where the programmer actually wants to treat a metric as a number. In Ayar this is intentional — such conversions should always be explicit and visible. Because no implicit coercion exists from metric to float (§4.7), and all operators require operand-type identity, any expression mixing the two types is rejected by the type checker.

5. Structured Type — The Experiment Record (§6.5–§6.7)

Ayar has one structured type: the **experiment record**. An experiment block accumulates trained model results, collected metrics, overfitting tags, a ranked ordering, and a best selection. This is a heterogeneous collection of named values — a record in Sebesta's terminology (§6.6).

Design decisions from Sebesta §6.5–6.7:

Field definition: Fields of the experiment record are not declared explicitly by the user. They are defined implicitly by the statements executed inside the block (`train`, `evaluate`, `collect`, `analyze`, `compare`). This is a deliberate DSL design choice — the user describes *what to do*, not *what shape the record has*.

Access: The experiment record is accessed via dotted notation from outside the block: `iris_exp.best`. Only the `best` field is externally accessible. All internal fields (individual model metrics, overfitting tags, ranked ordering) are encapsulated and not directly readable by the user — consistent with the stack-dynamic lifetime declared in §4.6.

Type equivalence for records: Name equivalence applies — two experiment blocks with identical internal structures are not interchangeable. `iris_exp.best` and `titanic_exp.best` are different bindings even if both contain a KNN model selected by `f1`. This prevents accidental substitution of results across experiments.

Lifetime: Stack-dynamic (Sebesta §5.8.2), as declared in §4.6. The record is created when the experiment block begins and destroyed when it ends. Only `best` survives via export to the global environment.

Names, Binding, Scope, Lifetime;

Legal Identifiers. In “ayar”, an identifier must begin with a letter (a–z or A–Z) and may be followed by any combination of letters, digits, and underscores. Underscores are not permitted as the first character — this is a deliberate restriction to keep identifiers visually clean and to avoid confusion with internal naming conventions. There is no length limit on identifiers. Keywords such as `dataset`, `model`, `train`, and `evaluate` are reserved words and cannot be used as user identifiers; the lexer classifies them before they reach the parser (Sebesta §5.2, reserved words approach). The same applies to metric literals (`accuracy`, `precision`, `recall`, `f1`) and boolean literals (`true`, `false`), which are recognized as dedicated token types rather than identifiers.

Binding and Binding Time. “ayar” distinguishes between bindings that occur at language design time and those that occur at runtime. The types of operators (`+`, `-`, `*`, `/`) and the set of valid model types (`KNN`, `DecisionTree`, `SVM`, `NaiveBayes`, `LogisticRegression`) are bound at language design time — they are fixed in the grammar and cannot be extended by the user. The association between a model name and its hyperparameter values (e.g., `knn_clf` bound to `{k = 3, distance = "euclidean"}`) occurs at runtime, specifically at elaboration time when the model declaration is processed. Similarly, the binding of a dataset name to its loaded file path happens at runtime when the dataset declaration is executed. In Sebesta's terminology (§5.4), these are dynamic bindings: the binding exists only during program execution and is established when the declaration is encountered during interpretation.

Scoping: Static (Lexical). “ayar” uses static scoping (Sebesta §5.7). There are exactly two scope levels in the current design:

- **Global scope:** All dataset, model, and experiment declarations live in global scope. A model declared at the top level is visible to every experiment block in the program.
- **Experiment scope:** The body of an experiment block creates its own inner scope. Statements inside the experiment (such as `train`, `evaluate`, `collect`, `analyze`, `compare`, `exclude`, `select`) can reference models and datasets from the global scope, but any names introduced inside the experiment (such as the implicit best result after `select best`) are local to that experiment.

The choice of static scoping is motivated by predictability and reliability (Sebesta §1.3.4). In an ML configuration language, the user must know exactly which model a `train` statement refers to by reading the source code — not by tracing the runtime call history. If Ayar used dynamic scoping instead, a model name inside an experiment could accidentally resolve to a different model depending on the order of execution, which would silently change training results. This would be catastrophic in an ML context where reproducibility is essential. For example, if two experiment blocks both declared a local model named `m`, dynamic scoping could cause the second experiment's `train m` to accidentally pick up the first experiment's `m` if scopes were not properly isolated. Static scoping eliminates this entire class of errors.

Lifetime. Local variables in Ayar — specifically, the internal state of an experiment block (trained model results, collected metrics, the best selection) — have stack-dynamic lifetime (Sebesta §5.8.2). They are allocated when the experiment block begins execution and

deallocated when the block ends. This means experiment results do not persist after the block closes, unless explicitly referenced by a top-level evaluate or report statement through dotted access (e.g., iris_exp.best). Global declarations (datasets and models) have static lifetime — they exist for the entire duration of the program, from the moment they are declared until the program terminates. There are no explicit-heap-dynamic variables in Ayar; the language does not provide manual memory allocation or deallocation, which is consistent with its goal of being a high-level DSL that abstracts away low-level concerns.

Expressions & Assignment;

- **Operator Precedence and Associativity (§7.3)**

Ayar uses stratified nonterminals to encode precedence directly in the EBNF grammar (Sebesta §3.3.1.8) — lower precedence operators appear higher in the grammar hierarchy, forcing higher precedence operators deeper into the parse tree.

Level	Operators	Associativity	Grammar Nonterminal
1 (lowest)	or	left	<expr>
2	and	left	<and_expr>
3	not	right (unary)	<not_expr>
4	== != < <= > >=	left, non-chaining	<rel_expr>
5	+ -	left	<add_expr>
6	* /	left	<mul_expr>
7 (highest)	unary -	right (unary)	<unary_expr>

All binary operators are **left-associative**, implemented via left-recursive while-loops in the recursive-descent parser (Sebesta §4.4.1). Unary operators (not, unary minus) are **right-recursive** — repeated applications such as not not x parse as not (not x). Relational operators are **non-chaining**: $a < b < c$ is a parse error, not a double comparison — this prevents the silent logical errors that chained comparisons produce in languages like Python.

- **Short-Circuit Boolean Evaluation (§7.6)**

Ayar does not use short-circuit evaluation — both operands of or and and are always fully evaluated. As Sebesta §7.6 notes, short-circuit semantics are primarily beneficial when operands have side effects. In Ayar, expressions have no side effects whatsoever, so skipping evaluation of the second operand would provide no practical benefit. More importantly, in constructs like exclude if overfit or underfit, both tags are pre-computed Boolean values — always evaluating both makes behavior completely predictable regardless of operand order, which directly supports Ayar's reliability priority (Sebesta §1.3.3). The trade-off accepted: if conditions ever become computationally expensive in a future extension, full evaluation will perform unnecessary work.

- **Order of Operand Evaluation (§7.3.5)**

Operands in Ayar are evaluated **left-to-right**, and this order is **fully defined** — not implementation-dependent. This is a deliberate reliability decision (Sebesta §1.3.3). Ayar has no side-effecting expressions (assignment is a statement, not an expression — see below), so in practice operand order never changes a result. However, making the order explicit in the specification ensures that future interpreter implementations cannot diverge on this point. Sebesta §7.3.5 notes that languages like Java also define left-to-right evaluation for exactly this reason.

- **Assignment: Statement, Not Expression (§7.7)**

Assignment in Ayar is a statement, not an expression. Assignment only appears in model field declarations ($k = 3$) and analyze fields (threshold = 0.08). It cannot appear inside an expression. There is no construct equivalent to C's $x = y = 0$ or `while ((line = read()) != null)`.

This is a direct writability sacrifice (Sebesta §1.3.2) in favor of readability and reliability (§1.3.1, §1.3.3). Assignment-as-expression is a well-known source of bugs — most famously the `if (x = 0)` vs `if (x == 0)` confusion in C. Since Ayar prioritizes readability and is used in a configuration context where silent errors are catastrophic (wrong model selected due to a misread assignment), the cleaner separation of assignment and expression is the correct trade-off.

Design Rationale;

- Separating metric from float (Name Equivalence)

Defining metric as a type distinct from float was one of the most critical design decisions for making Ayar work the way I intended. In the context of ML model selection, a threshold value of 0.06 and an accuracy score of 0.99 look numerically identical but are semantically entirely different. I used structural equivalence, these two values would be interchangeable, causing to unseeing errors that cause the wrong model to be selected — exactly the kind of mistake Ayar is designed to prevent. Name equivalence draws a hard line between them. The trade-off I accepted is that a user who genuinely wants to use a metric value in a numerical computation cannot do so directly; there is no explicit conversion mechanism, which makes the language more restrictive for that specific use case.

- **Strong Typing**

Designing Ayar as a strongly typed language is a direct consequence of its reliability priority. Assigning a value of the wrong type to a model field, or using a string in a numerical comparison, produces errors that are extremely difficult to detect in an ML configuration context — and anyone who has worked with ML pipelines knows that a single misread value can cost hours of debugging to trace back. Letting type errors pass silently was never an option. Explicitly reporting them tells the user exactly what went wrong. The trade-off I accepted is reduced flexibility and writability: for instance, I had to define an explicit coercion rule for `int` $\rightarrow$ `float` rather than letting the language silently accept any mixture of numeric types.

- **Encapsulating the Experiment Record**

The internal structure of an experiment block — individual model metrics, overfitting tags, the ranked ordering — is not accessible from outside the block. Only `exp_name.best` is exported to the global environment. The reasoning behind this decision is straightforward: in an ML configuration language, what the user needs to see is the result, not the process. Allowing intermediate metrics to leak outward would make user code dependent on the internal details of

the block, breaking any future changes to how those details are computed. The trade-off I accepted is that a user cannot access a specific model's validation accuracy from outside the block, which makes certain debugging scenarios harder — though the intention of Ayar is precisely that good results are obtained without needing to debug intermediate steps.

- **Assignment as a Statement**

Designing assignment as a statement rather than an expression was a deliberate decision in favor of readability and reliability. In C-style languages, errors like `if (x = 0)` are common and insidious. Ayar's target users are ML researchers, not programming language specialists, and a confusion between assignment and comparison in this context could directly cause the wrong model to be selected. Keeping the two constructs syntactically separate eliminates this class of errors entirely. The trade-off I accepted is reduced writability: chained assignment and assignment inside expressions are not possible, closing off some shortcuts.

- **Full Evaluation (No Short-Circuit)**

The decision to always fully evaluate both operands of `or` and `and` follows directly from the fact that expressions in Ayar have no side effects. Short-circuit semantics derive their value from situations where evaluating the second operand has a side effect that should be skipped — no such situation exists in Ayar. Furthermore, in constructs like `exclude if overfit or underfit`, both tags have already been computed before the expression is reached; tracking which condition was or was not evaluated would introduce unnecessary cognitive load with no benefit. The trade-off I accepted is that if conditions ever become computationally expensive in a future extension of the language, full evaluation will perform unnecessary work.